Encargado de Logística
Frontend (RutaDetallePage)
logistica-service
logistica_db (PostgreSQL)
EventosOutbox (logistica_db)
OutboxPublicadorLogisti-caService
Google Cloud Pub/Sub
Inventarios / Almacén (Servicio Externo)
PubSubResultadoSalidaC-onsumer
EventosInbox (logistica_db)
Fase 1: Transacción atómica de despacho y registro en Outbox (ABC-385 / ABC-386)
Confirmar despacho de ruta con boletas
1
POST /api/logistica/rutas/:id/despacho
2
BEGIN transacción
3
SELECT rutas FOR UPDATE · SELECT paradas FOR UPDATE · SELECT boletas FOR UPDATE
4
Valida máquina de estados: PLANIFICADA → DESPACHADA (boletas EMITIDAS)
5
UPDATE rutas SET estado = 'DESPACHADA', fecha_salida_real = NOW(), inventario_estado = 'PENDIENTE'
6
INSERT eventos_outbox (eventId A, inventarios.salida.v1, correlationId C, items_despacho, estado PENDIENTE)
7
COMMIT (cambio de estado de ruta + outbox en el mismo commit)
8
200 OK (Ruta DESPACHADA, salida registrada)
9
Fase 2: Publicación confiable desacoplada vía Outbox (ABC-386 / ABC-388)
SELECT eventos_outbox FOR UPDATE (lease transaccional PROCESANDO)
10
Publica DespachoRutaRegistrado.v1 (topic PUBSUB_LOGISTICA_TOPIC_ID, eventId A, correlationId C)
11
alt
[Broker confirma publicación]
messageId de Google
12
UPDATE eventos_outbox SET estado = PUBLICADO, messageId, publicado_en = NOW()
13
[Fallo transitorio o desconexión]
UPDATE eventos_outbox SET estado = PENDIENTE, reintentos++, proximo_intento (backoff)
14
Reintento conserva estrictamente el mismo eventId A y payload inmutable
Fase 3: Procesamiento en el dominio de Inventarios / Almacén
Entrega evento DespachoRutaRegistrado.v1 (at-least-once)
15
Valida disponibilidad física de lotes y stock reservado
16
alt
[Stock físico suficiente en almacén]
Registra salida de almacén y decrementa stock
17
Publica SalidaInventarioResultado.v1 (resultado: ACEPTADA, correlationId C, requestEventId A, eventId B)
18
[Discrepancia o faltante en almacén]
Publica SalidaInventarioResultado.v1 (resultado: RECHAZADA, motivo: 'Stock no disponible', correlationId C, requestEventId A, eventId B)
19
Fase 4: Consumo asíncrono e idempotencia con Inbox transaccional (ABC-387 / ABC-388)
Entrega SalidaInventarioResultado.v1 (al menos una vez)
20
Valida contrato v1 y origen del resultado de inventarios
21
alt
[Payload no es JSON o esquema inválido]
Registra advertencia permanente (RESULTADO_INVENTAR-IO_RECHAZADO)
22
ACK inmediato (sin bucle de reintentos)
23
[Evento válido]
BEGIN transacción
24
SELECT eventos_inbox WHERE event_id = B
25
alt
[Mensaje duplicado ya recibido]
ROLLBACK (sin cambios en datos)
26
ACK (idempotencia garantizada)
27
[Mensaje nuevo]
SELECT rutas WHERE id = C FOR UPDATE
28
UPDATE rutas SET inventario_estado = ACEPTADA | RECHAZADA (ABC-387)
29
INSERT eventos_inbox (eventId B, correlationId C, PROCESADO)
30
COMMIT
31
ACK solo después del COMMIT exitoso
32
Ante fallo transitorio de PostgreSQL: NACK y Pub/Sub reentrega sin pérdida